Smart Device for Visually Impaired

Kyshawn Savone-Warren, Luke Liu, Carlos Alberto Morelos Escalera, Santos Zaid, Nathan Cannell

Introduction

Our device is designed to assist individuals with visual impairments, including those who are blind or legally blind, by providing real-time object detection and distance estimation. While primarily aimed at enhancing mobility and awareness for visually impaired users, its capabilities also make it a valuable addition to augmented reality systems, such as Meta's Quest 3, where object recognition can improve user interaction with the environment.

Objectives

> Provide real-time object detection and distance estimation to assist visually impaired users in navigating their surroundings safely.

> Develop a compact system that can be easily mounted on a helmet or glasses without adding excessive weight.

>Deliver immediate auditory or haptic alerts based on detected objects and distances, allowing users to react quickly.

Device Configuration

For the creation of this device we combined the following objects onto a helmet:

> **Raspberry Pi**
> **STM32**
> **Arducam ToF Camera**
> **USB Wide Angle Camera**
> **Button**
> **Earbuds**
> **Battery pack**

We attached the Raspberry Pi and STM32 on the top, closer to the middle of the device. We also attached the two cameras, ToF and USB, to the front of the device so that they can have full visibility. The button was attached on the side in an easy to reach position. The earbuds would also run from the STM32 to play the audio cues. Lastly the battery was placed in the wearer's pocket.

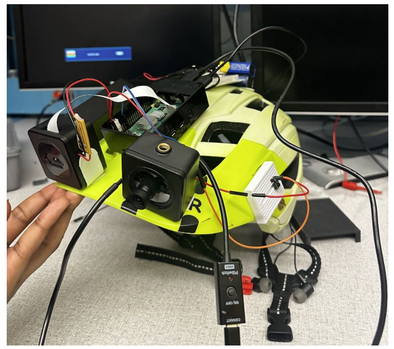


Figure 1: Picture of the device from the side view

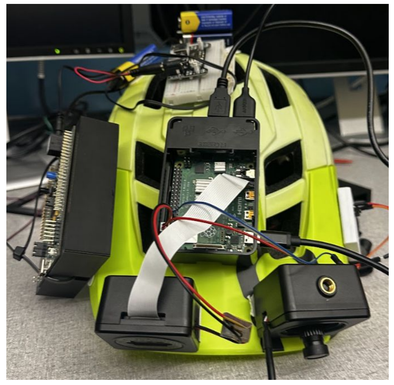


FIgure 2: Picture of device from front/top view

Navigation - Methods

For the navigation side of our project, we wanted it to be able to run on our wearable, while also not allocating too many resources. Additionally, we wanted it to run locally so that the user would not have to worry about connectivity issues. For these reasons we implemented OSMnx for our path planning algorithm. This allows us to:

> Have a offline path planning algorithm that can run on the Raspberry Pi from anywhere

> Lightweight, allowing for it to run simultaneously with other scripts on the Raspberry Pi while also not stunting their performance.

> Also allows for full customization of the program so that we could add things like realtime GPS location and walking velocity if we wanted too.

```
Continue on West Stevens Way Northeast for 15 meters Waiting
17.22 seconds before next navigation step...
Turn right onto 15th Avenue Northeast and continue for 69 met
ers Waiting 77.06 seconds before next navigation step...
Continue on 15th Avenue Northeast for 26 meters Waiting 29.59
 seconds before next navigation step...
Continue on 15th Avenue Northeast for 70 meters Waiting 79.08
 seconds before next navigation step...
Continue on 15th Avenue Northeast for 165 meters Waiting 185.
07 seconds before next navigation step...
Turn left onto Northeast 42nd Street and continue for 90 mete
rs Waiting 100.94 seconds before next navigation step...
```

Figure 3: Printed Navigations provided from gps

Object Avoidance - Methods

For our Object Avoidance, we decided to use an Arducam ToF camera. This type of camera allows us to determine object distances at a pixel level. How it works is by measuring the time it takes for a light signal to travel from the camera to an object and back. This method allows us to:

> Determine distances at a pixel level,allowing us to detect different objects at different levels. Which is useful for cases in which a very thin, low, or high object approaches the wearer.

>Function similarly to a lidar sensor but is less cost and more accurate, considering we only care about what is in front of the wearer.

>Easier to mount, we can utilize it's camera shape to mount next to the wide angle camera, allowing us to save room on the helmet for the Raspberry Pi and the STM32.

Object Detection - Methods

For Object Detection, we chose to use the YOLO (You Only Look Once) model. This deep learning based object detection framework allows us to identify and classify objects in real time. This method allows us to:

> Detect multiple objects simultaneously, ensuring objects such as traffic signs, stop signs, and other street cues are recognized.

> Operate efficiently on embedded hardware, as YOLO's optimized architecture ensures low-latency inference, making it possible to run on a system powered byu a Raspberry Pi.



Figure 4: YOLO object detection

Results

For our device demonstration, we successfully showcased all of its key functions, including real-time object detection, distance estimation, and auditory feedback. The system was able to recognize obstacles, traffic signs, and other important street cues while providing distance-aware alerts to the wearer. However, the demonstration process was not without challenges, and we encountered several roadblocks along the way.

> When transitioning from a wall adapter to a battery pack for portability, we faced unexpected power instability. The system would occasionally fail to deliver consistent performance, requiring troubleshooting and adjustments to our power management.

> While testing the auditory feedback system, we noticed a persistent static noise in the headphones, making the experience uncomfortable for the wearer. This interference required additional filtering and adjustments to ensure clear and effective audio output.

Conclusion

While our device successfully demonstrated real-time object detection and distance estimation, there is still room for improvement. Moving forward, we aim to refine the form factor, making the system more compact and comfortable for everyday use. Additionally, we plan to address the power stability issues, improve audio clarity by eliminating static noise, and optimize overall performance for a smoother user experience. By tackling these challenges, we hope to create a more practical and reliable assistive device for visually impaired users and potential AR applications.